Errata in AI-Executed Research: A Census of Six Papers, a Blind Replay of the Checks, and What the Record Says About Why

Roberto I. Ono Filho
Independent researcher
ono.roberto@gmail.com ORCID 0009-0006-8650-629X

Report, version 1, 22 October 2026. Not peer reviewed. Zenodo: 10.5281/zenodo.23562924.

Abstract

In August and September 2026, an AI assistant (Claude, inside Claude Code) wrote the code, ran the experiments and drafted the papers of a research program directed by one person. An audit on 5 October found errata in all six of its documents (four public preprints, one submitted to a workshop, one draft report). We froze the versions that existed before the audit and replayed, blind, four kinds of check against the 80 primary errata: a document-only review by six model families (two runs each), an audit of each paper against a snapshot of the project's records as they were when it was frozen, a mechanical number check and a citation check. Matching and adjudication were done by models too, and no step was validated by a person. One fresh review by Claude Opus 5.5 catches 35–38% of the audited errata (about half of them through findings it rated of low severity), among them the five numerical inconsistencies a human workshop reviewer had found, and 0.93–0.96 of its high- and medium-severity findings are real errors: about half matched audited errata, and most of the others are new errors that two adjudicating models, Opus 5.5 itself and GPT-6.1 Sol, both confirmed. The twelve reviews together catch 60%, and all checks together 71%. What the checks miss is mostly what the text alone cannot show: in a post hoc coding of each erratum's status on the day of its freeze, the twelve reviews caught 80% of the errata decidable from the document and 30% of those that needed the records of the time, and the records-level audit, in one budget-capped run per document, 30%. Claims about the work itself (that an AI assistant did the work, that a count was registered, what a companion paper now says) are mostly of the second kind. A strict post hoc re-check of the matching, by two blind model judges, keeps 87% of the matcher's credits (an erratum credited to a check); under it one Opus review catches 33% of the audited errata and 4 of the human reviewer's five, and the twelve reviews 71% and 10% of the two kinds. The replay also found about 70 errors the audit had missed, and Chao's capture–recapture estimator, best read here as indicative, puts the total at about 169 to 213 errors, depending on how the occasions are counted. Pilots on one synthetic study, in four presentations of up to 164k tokens (the largest with GPT-6.1 Sol only), then tested the writing step: given the whole record, Claude Opus 5.5 and GPT-6.1 Sol between them reconciled a stale draft with the final state in 11 papers without an error in the twelve claims scored in each, by an unvalidated automated scorer. These pilots, a floor result with later models, cannot show why our errors happened. They are consistent with the reading the assistant recorded in the notebook after them, on 6 October, from its dated entries of August and September: the errors came from the process, with papers frozen before the record settled, intentions written as facts and no reconciliation pass before deposit; the notebook itself also carried some of the errors. Three working rules follow.

1 Introduction

Research agents now write code, run experiments and draft papers, and the bottleneck has moved from producing research to checking it: their claims are harder to verify than their code is to run [8]. Published AI papers already carry several objective mistakes each [4]; AI-written papers add hallucinated numbers, references and experiments [3, 5, 14, 15]; and checkers, human or automated, catch a minority of known errors in most evaluations [2, 26, 29, 32], though recent agentic reviewers catch more of the errors injected into papers on purpose [17]. What is still missing is an account, on a real program, of which errors reach publication and which checks would have caught them. The case studies of human-directed agents that exist do not replay checks on their errors or measure what each would have caught [1, 18, 27]; the closest logs the errors an agent introduced in one program and how each was caught, and leaves unknown how many no check caught [25].

This report gives that account for one program. From August 2026, an AI assistant executed a research program on creativity and search in language models under the direction of one person, the author. The six documents studied here, frozen between 19 August and 27 September, are four preprints, a report and a short workshop paper [19–24]; a fifth preprint, a synthesis, was audited only for errors inherited from papers 1 and 2 and is not part of the corpus. The program used pre-registration, single reads and death criteria, and kept a dated notebook and its code, documents and analyses under version control (the raw run data lived outside the repository). On 5 October an audit of the six documents, again done with the assistant, found errata in every one. We use that record to ask three questions: which kinds of error got through; how much each kind of check, replayed blind on the versions frozen before the audit, would have caught; and why the errors happened.

The answers, in short. A single fresh review by a strong model catches about a third of the audited errata, including all 5 inconsistencies a human reviewer had found by the matcher's credits (4 under a strict re-check), and almost all of its high- and medium-severity findings were confirmed as real errors, by an adjudication in which the same model was one of two judges. Most of what the checks miss is what the text alone cannot show, and claims about the work itself are mostly of that kind. And in the pilots of Section 5, eleven papers written from one synthetic study, two current models given the record made none of these errors when they updated a draft. This cannot show why our errors happened, but it is consistent with the reading the assistant recorded in the notebook after the pilots, which places them in when and how the papers were written and frozen rather than in the writing itself.

2 The program and its record

The assistant wrote and ran all code, operated the data collection and drafted every paper; the author designed, decided and read. Per the commit trailers that carry one, the assistant models were mainly Claude Fable 5 (August) and Claude Fable 5.1 (September), with Claude Opus 5 for part of the draft report. Papers were written while batteries were still running and were frozen for deposit soon after the last read. Five rounds of review of one paper and two of another were done by AI reviewer agents; one human colleague reviewed the drafts of papers 1 and 2; and two anonymous reviewers of a NeurIPS 2026 workshop reviewed the short paper, which was rejected for capacity with scores of 7 and 6.

The audit of 5 October read each paper against the notebook, the appendices and the public releases, and checked the data for the items that changed a central claim. It found errata in all six documents. Some changed a declared finding: a claim that no arm produced a candidate beyond a classic baseline was false in the replication, and a diversity effect said to hold everywhere held in eight of nine problems. Others ran across papers: data-availability

statements were false in four papers, and reviews by AI agents were called "independent review" in one paper and left undescribed in another. All documents were corrected and deposited on Zenodo the same day [19–24]; the corrected papers 1–3 were submitted to arXiv as replacements on 22 October.

3 Method

The study was pre-registered (pre-registration v1) before any replayed check saw the frozen documents; six operational amendments followed, each with its reason (Section 9), and when each was written is said below. Version 1 had already changed the design of the draft version 0 in nine points that came from the implementation, before any model call on project text: among them the records-level audit, cut from two models with 60 tool calls and 250 thousand tokens each to one model with 40 calls and two million input tokens; a tenth class, METHOD; and the exclusion of errata that only appear in the compiled PDF. The taxonomy was written after the audits had been read. Amendments 1 and 3 were committed before the steps they changed. Amendment 2 was written as the replay began, which started while the matcher's validation set was still being generated; only the status, counts and cost of its cells were looked at until the matcher passed. Amendments 4–6 were written as or after their steps ran and were committed together with the results; with amendments 5 and 6 the matching script changed after the hash recorded at amendment 3; its final SHA-256 begins `8ec4d60f1630bf05`, and the deposit's README gives it whole.

Corpus. The six versions as they stood before the audit: paper 1 v1, paper 2 v1, paper 3 v3 and paper 4 v3, which were public, the short paper as submitted to the workshop, and the report's draft 0.4, not yet deposited. Each was exported from its git commit, flattened, with LaTeX comments blanked so a checker sees the text a reader of the PDF sees (not the figures or the rendered bibliography), and with its bibliography file appended.

Reference set. The errata of the 5 October audit and the five inconsistencies of the workshop reviewer, one item per defective statement, anchored by file and line in the frozen version (7 items are omissions, with no line): 86 items, of which 80 enter the primary analysis (3 that the audit itself flagged as of low confidence and 3 that only appear in the compiled PDF are kept out). Three minor items of paper 3 stay in, although the audit headed them "Menores (baixa)" (minor, low) on the scale it applied as confidence; when the set was fixed, before the replay, we read that label as their importance. Without them (77 items), one Opus review catches 34% and 35%, the twelve reviews 58% and all checks 70%. The audit judged each item against the record of 5 October; whether each was already wrong on the day of its freeze was coded post hoc, during the review of this report (Section 4.3). Each item has one of ten classes (Table 2) and a severity: G1 changes a central or declared finding (8 items), G2 changes a secondary claim or a number by more than rounding (39), G3 is minor (33). The coding is by the assistant, the model family that wrote and audited the papers; a second coder from another family (GPT-6.1 Sol) agreed on the class in 88% of items ($\kappa = 0.85$) but only in 55% on severity ($\kappa = 0.25$), so severity results carry that caveat; as registered, the second coder's classes and severities enter a sensitivity analysis (Sections 4.2 and 4.7).

Checks replayed. No checker saw the reference set, the audit or any later version, and no model checker had web access; the citation check queried the arXiv and Crossref services.

- **C1, document-only review**: a fixed prompt, written after the audit (its categories chosen with the audit's findings known, which if anything favours the checks), that names

the kinds of error without naming any erratum, asking for every error (numbers inconsistent with tables or appendix, unsupported or overbroad claims, contradictions, false statements about the work's status, data or review, citation problems), answered by six families: Claude Opus 5.5, GPT-6.1 Sol, Kimi K3, DeepSeek V3.2, Mistral Large 3 and Qwen3-Next-80B, two runs each, on Amazon Bedrock, at each provider's default sampling settings; the two runs are independent calls with the same prompt. The Claude calls used adaptive extended thinking and prompt caching; the other families ran with their default reasoning behaviour.

- **C2, records-level audit**: the same prompt plus read-only tools (list, read, search) over a snapshot of the repository at the version's commit, without reviews, audits, strategy, funding, field notes or publishing files (the release recipe among them) or the human rating packs, scrubbed of personal data (so an excluded file looked absent to it); Claude Opus 5.5, one run per document, at most 40 tool calls and two million input tokens counted cumulatively, cache reads included. Of the six audits, 3 reached the token cap after 20 to 29 calls and were asked to answer from what they had read; since every call re-reads the context, they had read at most about 109 to 151 thousand distinct tokens each (uncached input plus what was written to the cache).
- **C3, mechanical numbers**: every decimal in the body looked up in the document's own tables and appendix (the draft pre-registration also named the repository's appendix files; the frozen script, registered by its hash, looks only inside the document).
- **C4, citations**: every bibliography entry with an arXiv identifier or DOI checked for title, author surnames and primary category. It ran on all six documents, partially on 4 of them (some queries met rate limits or service errors), and 21 of its 71 findings, all counted in its precision, are identifiers the arXiv API did not return.

Matching and adjudication. Each finding was matched to the reference set by a model outside the checkers (Claude Sonnet 5.5) with a fixed rubric: same statement and same defect, at any of the erratum's locations and whatever correction the finding proposed. The matcher was validated before use on 30 synthetic paraphrases of known errata and 30 synthetic findings raising a minor, different concern about the same statement (recall 1.00, precision 0.97); these were easier than the near misses the draft pre-registration described (the same passage with another defect, or the same defect elsewhere), and the validation did not test a substantive different concern about the same sentence, the confusion the strict re-check below found. Of 30 real matches described at the time as drawn at random (the sample is archived; how it was drawn was not recorded), 28 were right when the assistant (Claude Opus 5.5) checked them one by one; no person checked the matches. Of the 72 C1 replies, 13 could not be read as strict JSON, all from the three weakest families: 11 were read by a format-only parser fixed in an amendment before matching, and the other 2 were matched as free text. The matcher left 205 findings without a decision; amendment 6, written after it had run, settled them from the format of its replies: 24 were recovered by re-reading them, 120 left out of complete replies were read as not matching (none from the Opus reviews or the records-level audit), and 61 were matched in one more pass. Of the 1246 findings, the 448 unmatched ones that their checker rated high or medium were grouped into 264 candidate errors and judged by two models (Claude Opus 5.5 and GPT-6.1 Sol) reading the document; a candidate counts as a new error when both say yes. Each answered yes, no or uncertain; they gave the same answer on 80% of candidates (134 both no, 71 both yes, 7 both uncertain), Opus said yes to 72 and GPT-6.1 Sol to 98, and 99 had at least one yes; the class and severity of a new error are the Opus adjudicator's. After a referee in the review of this report read some credited findings as raising a different concern about the same sentence (Appendix A), every match to a primary erratum (226 pairs of a finding and an erratum) was judged again, each pair on its own and with the matcher's rubric (in one call

Table 1: Recall of each check on the 80 primary errata, and precision on the high- and medium-severity findings: a finding is a real error when it matched an audited erratum or was confirmed as a new error by both adjudicators (Claude Opus 5.5 and GPT-6.1 Sol); "matched" counts only the first kind, and "strict" keeps only the matches that survive the strict re-check of Section 3, counting a finding whose match did not survive as false (a lower bound, rounded down). Two runs per family in C1, and "either" is their union; for C3 and C4, whose findings are all of low severity, precision is over all findings and a lower bound, since their unmatched findings were not adjudicated.

check	run 1	run 2	either	precision	matched	strict
C1 Claude Opus 5.5	28 (35%)	30 (38%)	35	0.93 / 0.96	0.52 / 0.52	0.88 / 0.92
C1 GPT-6.1 Sol	26 (33%)	18 (23%)	26	0.75 / 0.78	0.25 / 0.23	0.72 / 0.75
C1 Kimi K3	18 (23%)	22 (28%)	25	0.69 / 0.86	0.41 / 0.50	0.68 / 0.77
C1 Mistral Large 3	5	3	6	0.11 / 0.06	0.09 / 0.04	0.05 / 0.04
C1 Qwen3-Next-80B	3	4	5	0.06 / 0.09	0.04 / 0.07	0.04 / 0.05
C1 DeepSeek V3.2	2	1	3	0.25 / 0.15	0.08 / 0.08	0.25 / 0.15
C2 Opus 5.5 with the records	30 (38%)	–	–	0.71	0.50	0.67
C3 numbers	6	–	–	≥8/276		≥4/276
C4 citations	2	–	–	≥2/71		≥2/71
C1, all twelve reviews			48 (60%)			
all replayed checks			57 (71%)			

per document for one judge, in one session for the other), by GPT-6.1 Sol and a fresh Claude Fable 5.1 agent, neither told which check produced a finding (though a mechanical check's findings show it in their form) and both blind to the freeze coding and each other. Under this strict credit a check keeps an erratum only if both judges say one of its findings identifies it. GPT-6.1 Sol accepted 193 pairs and the Fable agent, which by its own account read release statements leniently, 217; they agreed on 89%, and every pair GPT-6.1 Sol accepted the Fable agent accepted too, so strict credit is GPT-6.1 Sol's verdict. It keeps 172 of the matcher's 198 credits (an erratum credited to a check), 87%. The re-check is post hoc, written down before the judges ran; the matcher's credits, with amendment 6, remain the primary result.

Analysis. Recall of each check on the primary reference set by class and severity, precision on high and medium findings, and Chao's lower-bound estimator [6] for the total number of errors, from which the number no occasion saw follows. Five predictions and a rule for continuing to a prospective study were registered. Recall by document for every check, the overlap between checks and the estimate by class, also registered, were left out of earlier drafts of this report and are in Appendix B. The replay cost US$44.44 at upper-bound per-token rates.

4 Results

4.1 One review catches about a third

Table 1 gives the main result. A single review by the strongest family catches about a third of the audited errata, and nearly all of its high- and medium-severity findings are real errors (Wilson 95%: 0.77–0.98 and 0.80–0.99): about half of them matched audited errata (14 of 27 and 13 of 25), and most of the others were new errors that both adjudicators confirmed (11 and 11), one of the adjudicators being Opus 5.5 itself. Recall and precision, though, count different findings: recall counts a finding of any severity, precision only those the checker rated high or medium. Of the 28 and 30 errata the two Opus runs caught, 15 and 17 were caught only through findings it rated low, which were never adjudicated (18 of its 51 and 18 of its 50 low findings matched an erratum); a reader who acted only on its high- and medium-severity findings would have caught 13 and 13, and over all its findings precision is at least 0.55 and

Table 2: Primary errata by class, and how many the twelve reviews (C1) and all replayed checks caught.

class	what it is	n	C1	all
INTERNAL	contradiction inside the document	16	16	16
NUM	number differs from the record or the appendix	15	10	13
SCOPE	claim broader than what was measured	15	9	12
META	false claim about artifacts, review or registration	14	7	7
MIX	two measures reported as one	7	4	5
TYPO	typographical error	4	1	1
METHOD	inaccurate description of what was done	3	1	1
STATUS	false statement about the progress of the work	2	0	0
PROP	error inherited from a companion paper	2	0	0
CITE	bibliographic error	2	0	2

0.56. The records-level audit caught 16 of its 30 only through low findings. Two more families reach 23–33% per run, with lower precision; three catch few errata, with low precision, and 13 of their 36 replies could not be read as strict JSON (Section 3). The records-level audit, with the same model and the repository of the time (notebook, appendices and scripts, not the raw run data), catches no more in total than a document-only review, but a different part: 11 items that neither Opus review caught, while it missed 16 that they did. Its precision counts one finding, the only one we found, that the snapshot's exclusions produced: the audit reported as missing the review reports that a paper said were archived, which had been left out of its snapshot (0.74 without that finding). Pooling every review and check reaches 71%. Under the strict re-check the two Opus runs catch 26 and 26, the twelve reviews 38 (48%), the records-level audit 28 and all checks 48 (60%). With 80 items, one run's recall carries a wide interval (Opus: 0.25–0.46 and 0.28–0.48, Wilson 95%); on the same errata the two runs disagree on 5 items one way and 7 the other (exact McNemar $p = 0.77$), so no difference between them is detectable at this size. Recall varies by document: the two Opus runs together caught 5 of the 6 errata in the short paper (26k characters as sent) but 6 of 14 in paper 1 (199k) and 4 of 17 in the draft report (140k).

4.2 What nothing catches

Table 2 shows where the checks fail. Every internal contradiction was caught by at least one review (15 of the 16 under the strict re-check). Claims about the work itself were not: of the 14 META items, the reviews and the records-level audit together caught 7, or 7 of 11 without the 3 omissions of an AI-use statement, errata only under a requirement the audit adopted and that no check was asked about; no check caught either STATUS item or either PROP item, and one of the STATUS items was defensible when its paper was frozen (Section 4.3). Of the 23 errata that no replayed check caught (20 without those omissions), 7 are META, 3 SCOPE, 3 TYPO, 2 MIX, 2 STATUS, 2 NUM, 2 METHOD and 2 PROP. The META, STATUS and PROP items among them are statements a reader cannot check against the text, for example three papers that do not say an AI assistant wrote their code and text, a count computed after the fact listed as registered, a design note described as having a registered stopping rule, released data said to hold a battery they lack, and what a companion paper now says.

By the first coder's severities, the records-level audit sees the central errors best: it caught 6 of the 8 G1 items, against 3 and 4 for the two Opus reviews; all checks together caught 7 (under the strict re-check 5, 3, 3 and 5). This does not survive the registered sensitivity analysis with the second coder's severities, which agree poorly with the first and count 17 G1 items: of those, the records-level audit catches 6, the two Opus reviews 5 and 6, the twelve reviews 11 and all

Table 3: Post hoc: recall on the 49 errata that were wrong at the freeze and decidable from the document, and on the 30 that were wrong at the freeze but decidable only from the records of the time, with the matcher's credits and with the credits that survive the strict re-check (Section 3).

	matcher		strict re-check	
check	document	records	document	records
C1 Claude Opus 5.5, run 1	28 (57%)	0 (0%)	26 (53%)	0 (0%)
C1 Claude Opus 5.5, run 2	27 (55%)	3 (10%)	25 (51%)	1 (3%)
C1, all twelve reviews	39 (80%)	9 (30%)	35 (71%)	3 (10%)
C2 Opus 5.5 with the records	21 (43%)	9 (30%)	21 (43%)	7 (23%)
all replayed checks	41 (84%)	16 (53%)	37 (76%)	11 (37%)

checks 13.

4.3 What the document can show

The audit read each frozen version against the record of 5 October, so an erratum may have been invisible to any check of its frozen version. During the review of this report two fresh agents (Claude Fable 5.1 and Claude Opus 5.5), blind to each other and to every result of the replay, coded each primary erratum from the frozen text, the git history, the notebook and the audit's description of the defect as wrong at the freeze and decidable from the document, wrong at the freeze but decidable only from the records of the time, or wrong only because of something that happened later. They agreed on 75 of 80 items ($\kappa = 0.87$); the 5 disagreements were settled on the records by the assistant that drafted this report, which knew the replay results (the reasons are archived). One of the second coder's codes was supplied on request, after its first answer left the item out, and the tallies in its note differ from its array, which is what we used. Read with each coder's own codes instead (the first, then the second), the twelve reviews catch 79% and 77% of the errata decidable from the document and 30% and 35% of the records-only ones, and the records-level audit 33% and 26% of the latter; under the strict re-check the twelve reviews keep 3 of the first coder's 30 records-only errata and 4 of the second's 31, against 3 of 30 with the settled codes. In all, 49 errata were decidable from the document, 30 needed the records and 1 became false later: the second human round, which was ready and waiting for raters when paper 1 called it in progress, and was never run; read instead as an intention written as a fact, it too would count as wrong at the freeze (31 needing the records). We keep the coders' code: "are released" could be checked as false on the day, while "in progress" could describe a round being prepared. Of the 30, 3 are omissions of an AI-use statement, errata only under a requirement the audit adopted; without them 27 need the records. This analysis was not registered; it was added after the single read.

Table 3 splits the main result along that line. A single Opus review catches more than half of what the document can show and little of what it cannot (0 and 3 of 30); the twelve reviews together catch 80% of the first kind and 30% of the second, or 71% and 10% under the strict re-check, which removes most of the reviews' credits for the second kind (3 of 9 survive; with either judge alone, 10% and 23%). One of the 3, a release claim of paper 1, survives through a free-text reply from Mistral Large 3 that reports a 404 error from the repository's address, a check a checker without web access could not have made, like the tool results never obtained that Gurram [10] describes; without it 2 of 30 survive (7%). The records-level audit had the repository of that date but not the raw run data, the git history or outside records; by a coding of the coders' evidence done by the same assistant, knowing the results (archived; 1 item was recoded in the fifth review round, when a referee showed that the audit had decided it from files in its snapshot), 24 of the 30 could be decided from its snapshot, and it caught 9 of those (a catch shows that the snapshot could decide the item, so every catch is among

them by construction). Its tool calls named the notebook in every run (1 to 18 times; 1 for the draft report, the document with the most errata, whose run also made 4 searches over folders that hold it). Of the 18 claims about the work itself, 3 were decidable from the text and all were caught; 14 needed the records and 4 of those were caught by some check; 12 of the 14 were decidable from the audit's snapshot, and the records-level audit itself caught 2 of them; 1 became false later. Of the 23 errata no check caught, 14 needed the records (10 of them claims about the work), 8 were decidable from the document (1 of them central, or 3 by the second coder's severities) and 1 became false later.

4.4 The human reviewer's errors

The short paper reached the workshop with five numerical inconsistencies, which one of its two reviewers found. A fresh Opus 5.5 review of the submitted version catches all 5, in run 1 (5) and in run 2 (5); GPT-6.1 Sol catches 4 and 3, Kimi K3 2 and 4, the records-level audit 2. The strict re-check of Section 3 rejects every finding paired with the fifth, an agreement range that mixed two samples, so under it Opus keeps 4 and 4, GPT-6.1 Sol 4 and 3, Kimi K3 1 and 3, and the records-level audit 2.

4.5 Errors the audit missed, and how many remain

The adjudication confirmed 71 errors that the audit of 5 October had not listed: 29 internal contradictions, 24 overbroad claims, 14 number errors and 4 others; 0 central (G1), 37 G2 and 34 G3. GPT-6.1 Sol found 59 of them (35 that no other check found), Kimi K3 18, Opus 5.5 16 and the records-level audit 6. Because GPT-6.1 Sol is also one of the two adjudicators, its share may be inflated by self-preference; each new error still needed the other adjudicator's agreement. The candidates were findings the matcher had left unmatched, and no second pass checked them against the audited errata. A referee in this report's review found that one, from Kimi K3's first run, is an audited erratum the matcher had missed (the short paper's claim that its effect ended about one point smaller, one of the human reviewer's five, so that run caught 3 of them, not 2), and that three restate parts of audit items the reference set kept as one item; so the replay found about 70 new errors.

Over the 151 errors of the primary set and the new ones, counted across all occasions (the audit, the workshop reviewer and every replayed check), 43 were seen by exactly one occasion and 47 by exactly two. Chao's estimator of a lower bound gives 170.7 errors (the 1987 form; with the factor $(t-1)/t$ for $t = 17$ occasions, 169.5), so by this estimate all of us together saw about 88% of them, and about 20 remain unseen. The capture histories are not alike: an audited erratum is caught by a matched finding of any severity, a new error only by a finding its checker rated high or medium, since low findings were never adjudicated, which adds to the new errors seen once. Chao's variance formula, which treats the occasions as independent, gives an approximate 95% interval of 160–194; the estimate is not a guaranteed bound. The occasions are not exchangeable: the audit defined the primary set, and over the replayed checks alone, which saw 128 of the 151 errors known, the same estimator gives 147, fewer than are already known: it counts only errors those checks could catch, and the 23 seen only by the audit or the workshop reviewer may lie outside their reach. With occasions this dependent, all of these estimates are better read as indicative than as bounds. The pre-registration's text defined the occasions per family (the audit, each model family of C1 with its two runs together, C2, C3 and C4: $t = 10$; another of its passages adds the workshop reviewer), while the frozen analysis script counted each run and the workshop reviewer; we did not notice the difference until the fifth review round. Per family (the audit as an occasion holding the 75 primary errata it found, and the workshop reviewer's five counting only where a replayed check caught them), 70 errors were seen once and 38 twice, and the estimate is 215.5 (interval 186–269; coverage 70%, about

64 unseen), or 217.2 with the workshop reviewer as an eleventh occasion. All of them estimate a lower bound, and counting per family gives the larger. They rest on the matcher's credits; with the strict ones, per run, the estimate is 181.0. With the duplicate above merged, they are 168.8 per run and 212.6 per family.

4.6 A second opinion

Is it better to run the strongest reviewer twice, or once together with another family? The comparison below is descriptive; with 80 items no difference is claimed. On the primary errata, two Opus runs catch 35; one Opus run with one run of another family catches 32.2 on average, 36.5 with GPT-6.1 Sol, 33.0 with Kimi K3 and 29.5–31.5 with the three weaker families, so Opus with GPT-6.1 Sol catches slightly more than two Opus runs, while the average over families catches fewer. Counting the new errors too, the average flips (51 against 54.1), driven by GPT-6.1 Sol (91.5), whose own findings it helped to adjudicate. Here a second opinion from another family paid off only when that family was strong for the task, as in the planted-error experiment of Song [30], where a top-tier cross-model reviewer added to a same-model review and a lightweight one did not, though there too the gain could not be separated from the second reviewer's own strength.

4.7 The registered predictions

Three of the five predictions failed, and the failures are informative. We expected document-only reviews to catch at most 20% of META and STATUS items because those need the records; pooled, they caught a proportion of 0.44 (0.54 without the omissions of an AI-use statement, 0.31 under the strict re-check). We expected the records-level audit to catch at least half of them; it caught 0.25 (0.31; 0.25 strict). We expected the mechanical number check to catch half of the NUM items with low precision; it caught 0.27, with a precision of at least 8 in 276 (about 0.03: the findings that matched an audited erratum). Prediction five held: no single check exceeded 50% recall (the best reached 0.38). Prediction four held only when the new errors are counted (above); the frozen analysis script, which also counted the items kept out of the primary set, scored it as holding (52 against 55.2). With the second coder's classes the first three come out the same (0.44, 0.25 and 0.25 on its 20 NUM items). Access to the records, used this way (one run, with a token cap that cut three of the six audits short), caught few claims about the work: 2 of the 12 that our post hoc coding finds decidable from its snapshot (Section 4.3). The pre-registration made a prospective planted-error study conditional on a class in which a cheap or records-level check beats the pooled reviews by 0.3 in recall; only CITE met that condition, a class of two items that the citation check caught and the reviews did not. The rule said when such a study would be justified, not that it would run; we judged two items too thin a basis and did not run it, and the pilots of Section 5 ask a different question.

5 What the record says about why

One reading of these results is that the writing step fails: a model updating a draft copies a superseded value or keeps a stale status. Models do bind stale values in long, changing contexts [9, 31]. Before testing writing rules on that premise, we piloted it on synthetic study records built with the traps that caught us: a draft written from a first read, a notebook with superseded runs, an analysis announced and cancelled, a release that left out the individual judgments, reviews by AI agents, a companion paper corrected later, a model run that differed from the plan, and a hypothesis registered after the data. Each record has twelve required claims with known truth, scored by Kimi K3. The pilots were run before the writing-rules study's protocol

was frozen, and it never was; the validation of the scorer that its draft planned (sixty claims labelled one by one by the assistant, blind to the arm, before any result was read) was not done, so the scores are an unvalidated model's, checked only by the assistant's reading of three papers.

The writers were Claude Opus 5.5 and GPT-6.1 Sol. The tasks were to make the draft consistent with the final record (4 papers), or only to "update the draft with the final results" (6), or, in one paper, to edit the prose while the harness filled the numbers and keyed claims from the artifacts. The records are one synthetic study, with the same values and the same twelve required claims, in four presentations: narrated (the truth written in the notebook), at 11–16k tokens as counted by the provider; the same narrated record with distractor entries added, at 80k and 164k tokens (Opus 5.5 returned an overload error four times on the largest, which was run with GPT-6.1 Sol only); or silent: the release contents only in a file listing, the reviewers' nature only in the reviews' headers, the model run only in a run configuration, and the cancelled analysis only by its absence.

Across 11 papers and 132 required claims (the same twelve in each paper) there were 0 errors and 0 omissions. Asked only to update the results, the writers also corrected the release, review, status, model, registration and companion claims; in the silent record they inferred from the absence that the analysis never ran, read the release listing, and recognised the reviewers as AI agents from the headers. The assistant read three of these papers against their scores. With the baseline already at zero errors, the full writing-rules study would most likely have hit the same floor, and was not run.

The pilots do not show why our errors happened: the papers were written by earlier models (Fable 5 and 5.1, with Opus 5 for part of the draft report) from a less legible notebook, in Portuguese, larger at most freezes than the records Opus 5.5 completed in the pilots (Section 8). What they show is that two current models, writing with a legible synthetic record in context, made none of these errors in 132 required claims (12 of them in the paper whose numbers the harness filled), and the notebook shows how the papers were actually written. The pilots are consistent with the reading the assistant recorded in the notebook after them, on 6 October, which places the errors in the process; the reading is post hoc, and the process facts it names come from the notebook's dated entries and the audit. One caveat: the record was not always right, and the notebook itself carried at least two of the errata (on 18 August it called the fresh-text surprise effect about one point smaller, and on 25 August the diversity effect positive in all nine cells). The process:

- papers were written while batteries were still being re-judged, and frozen before the record settled, so provisional numbers stayed in the body while the regenerated appendix carried the final ones;
- intentions were written as facts ("are released", "is in progress") and never revisited when the release went out without the judgments or the second human round was dropped;
- no reconciliation pass against the final record was run before deposit or announcement;
- companion papers were updated one at a time, so errors propagated between them.

The records-level audit saw each frozen version with the repository of its own date. By our coding, 12 of the 14 claims about the work that needed the records could be decided from that snapshot, though it caught only 2 of them, and 2 rested on artifacts it did not have, the contents of a release and the git history; one more, the human round described as in progress, became false only after the freeze.

6 Three working rules

The rules below come from one program and are offered as hypotheses for others to test, not as findings.

1. **Do not write an intention as a fact.** "Will be released" until the release exists; "planned" until the run has run. A claim about the work's own state is true only when written, so it must be written after the fact.
2. **Reconcile before every deposit, and again after any change.** Give a strong model the paper and the whole final record and ask for the version consistent with it; in our pilots, writers given the whole record of one synthetic study made no errors in the 132 required claims scored (the same twelve in each paper), 48 of them under this instruction (automated scoring; the assistant read three of the papers). The pass is only as good as the record, and here the notebook itself carried some of the errata. Repeat it for every published paper that a later change touches, companions included.
3. **Run a fresh review before submitting.** One review by the strongest available model, on the document alone, would have caught about a third of the audited errata (about half of them only through findings it rated of low severity, so all of its findings should be read), more than half of those the document itself could show, and all 5 inconsistencies the human reviewer found by the matcher's credits (4 under the strict re-check), with almost all of its high- and medium-severity findings confirmed as real errors, by an adjudication in which the same model was one of two judges. Add a second strong family if one is available; a weak one adds noise.

Generating every number from the artifacts is the design of data-to-paper [11]; locking claims to approved evidence removed fabricated citations in a controlled test, with preliminary evidence for reported results [12], ResearchLoop admits claims to a paper through a ledger bound to evidence [33], and deterministic checks caught seeded defects [16]. This report's numbers are generated from the archived evidence. But construction only passes on what the artifacts say: it helps where a manifest of the work's state exists and is kept current, which in this program it was not.

7 Related work

Errors in published AI papers have been counted with automated checkers [4]; errors in AI-written papers with reconstruction and audits [3, 5, 14, 15]; and the verification gap of autonomous research agents has been surveyed [8]. Benchmarks of error detection report low recall for automated reviewers [2, 29, 32], though on errors injected into real arXiv papers the best agentic review system caught 71.6% and the union of six models 83.3% [17], AI reviewers whose criticisms overlap far more than human reviewers' do [13], a benefit from a strong second model, but not a lightweight one, on planted errors [30], and, for human reviewers, about 2.6 to 3 of 9 planted major errors found [26]. Agents overclaim what they did [28] and assert tool results they never obtained [10]. In the automated alignment researchers of Chen et al. [7] the agent does not write its own scores: a separate evaluator produces them; evidence-locked writing removed fabricated citations in a controlled test [12], ResearchLoop admits claims to a paper through a ledger bound to evidence [33], and deterministic gates catch seeded defects that a single LLM review misses [16]. Human decision gates, together with deterministic data work, cut critical failures in AI-assisted social science [34]. The single-investigator case studies closest to ours report events and the practices that caught them, without replaying checks or measuring what each would have caught [1, 18, 27]. Closest of all, Orden [25] logs 58 errors

an AI agent introduced in one research program over 51 days, coded by the mechanism that caught each and how far it had travelled: no single mechanism caught a majority, an audit by another agent of the same family caught the most, and in four the archive itself carried the error forward, as our notebook did. It counts only errors that were caught, so it cannot say what each check would have missed or how many no check caught. What this report adds is a census of the errata in six documents of one program, four of them public, with a dated notebook and its code and texts under version control, tied to a blind replay of the checks, with an estimate of what everyone missed and a test of the writing step on synthetic records.

8 Limitations

This is one program, one assistant family and six documents, not a sample of AI-assisted research. The reference set comes from an audit by the same family that wrote the papers, and the coder, the matcher and one adjudicator are from that family too; the other families and the capture–recapture estimate soften this without removing it, and the precision of the strongest reviewer rests in part on new errors that its own model helped to confirm. Severity coding agreed poorly across coders. No step was validated by a person: the reference set, its coding, the check of the matcher, one adjudicator and the reading of the pilot papers are model work. The coding of each erratum's status at its freeze is post hoc and by two model agents of the same family as the audit, the matcher and one adjudicator, with their disagreements settled by the drafting assistant. Document-only reviews were credited with 9 errata coded as decidable only from the records; the strict re-check keeps 3 of them, so most of those credits came from a generous matcher, and the re-check's judges are models too, which accepted one credit resting on a check its checker could not have made. The matcher's recall on real findings was not measured. In the assistant's check of the matcher, the 13 sampled matches of Opus findings were all right, and 15 of the 17 of the other checks' findings were. The direct test of the writing step, giving the reconciliation pass the real frozen drafts with their own snapshots, was not run. Precision is not comparable across checks, since it counts only matched or doubly adjudicated findings, C3 and C4 were never adjudicated, and its denominator is each checker's own severity labelling, which varied from 34% to 94% of a family's findings rated high or medium (Opus 34%, GPT-6.1 Sol 85%). The records-level audit ran once, with a tool and token budget that cut three of its six runs short, and without the raw data. The synthetic records are one study, the same twelve facts in four presentations, legible in a way real notebooks are not; their scoring is automated (Kimi K3), with three papers read by the assistant, the pilot writers are later models than the ones that wrote the papers, and records beyond 164k tokens as counted by the provider were not tested. Counted with one tokenizer (cl100k), the largest pilot is 165k and our notebook measured 35k to 187k at the six freezes, so the largest pilot covers its size at 5 of them for GPT-6.1 Sol and at 2 for Opus 5.5 (whose largest completed record, 80k tokens by the provider's count, is 56k in cl100k), though not its legibility or its language (Portuguese). Papers 1–3 were public on arXiv, and the public mirror of the repository carried the notebook to 28 August, before the replay; the checkers had no web access, but whether these texts were in their training data is unknown. The author directed the work under study, and the assistant that drafted this report is a later model of the family whose errors it counts.

9 Availability, use of AI and acknowledgements

The pre-registration (v1 and its six operational amendments), the audit of 5 October (the consolidated document and the five per-paper audits the reference set cites, in Portuguese), the reference set, the scripts and the evidence of the replay and of the pilots (raw responses, some redacted as said below, with the size of each request, matches, adjudications, captures, analyses,

SHA-256 manifests), the post hoc coding of Section 4.3 and the archive of this report's review rounds are in the author's research repository, which is private, and the frozen versions of the six documents as the checks saw them (flattened texts with their line maps and bibliography files) were exported from its commits by `scripts/r1_freeze.py`, so the deposit is where these flattened texts become public (four of the documents were public already, the short paper as submitted and the draft report were not); all of it, the frozen versions included, is deposited with this report, with the final hash of every script; the requests of the document-only and mechanical checks are rebuilt by the scripts from the frozen documents (for paper 1 the deposited copy is redacted, so its rebuilt requests differ slightly from those sent), while the records-level audit's also held what its tools returned from the snapshots: for those the logs keep each call and the size of its result, and the snapshots are not deposited (below). In the deposited copies, the per-rater statistics of paper 1 v1 (each rater's mean and agreement with the judge, which with the raters' profiles in order identified each rater, and the agreement between each pair of raters on surprise) and the payment values are redacted, as in its corrected version (which gives the statistics as ranges), there and wherever a scan of everything deposited, raw logs and the review archive included, finds them: in five checker responses (in one of them, Kimi K3's first run, only in the reasoning its raw log keeps) and the files derived from them; a colleague's name in two of the audit documents, the workshop reviewer's anonymous handle and a private term in one script are redacted too, and a list (`REDACTIONS.json`) gives the hash of each redacted file before and after. The records-level audit's snapshots of the repository are not deposited, since they hold the whole repository of those dates, the notebook included (the audit's responses quote some of its lines); their SHA-256 manifest is, but the records-level audit cannot be rerun from the deposit. The freeze script also depends on a private file of scrubbing rules, so the deposited flattened texts and the snapshots' manifest, rather than the script, are what a reader can reuse. Individual human ratings from the series remain unreleased to protect rater anonymity.

Before deposit, this report was itself put through rule 3 in rounds of review; Appendix A lists what each found; every finding was corrected here or answered in the archived triage of its round.

The replay, the pilots and this report were produced by the author working with an AI assistant (Claude, Anthropic; model Claude Opus 5.5, October 2026) inside the Claude Code environment: the assistant wrote and ran the code, operated the runs under the author's direction and drafted the text; the author decided the questions and the design and is responsible for the content. The checker, matcher and adjudicator models were run on Amazon Bedrock. Thanks to the colleague who read the series' drafts, and to the workshop reviewer whose five findings are the human baseline of this report.

References

[1] Anas H. Alzahrani. Persistent AI agents in academic research: A single-investigator implementation case study. arXiv:2605.26870, 2026.

[2] Tim Baumgärtner and Iryna Gurevych. SciCoQA: Quality assurance for scientific paper–code alignment. arXiv:2601.12910, 2026.

[3] Joeran Beel, Min-Yen Kan, and Moritz Baumgart. Evaluating Sakana's AI scientist: Bold claims, mixed results, and a promising future? arXiv:2502.14297, 2025.

[4] Federico Bianchi, Yongchan Kwon, Zachary Izzo, Linjun Zhang, and James Zou. To err is human: Systematic quantification of errors in published AI papers via LLM analysis. arXiv:2512.05925, 2025.

[5] Federico Bianchi, Owen Queen, Nitya Thakkar, Eric Sun, and James Zou. Exploring the use of AI authors and reviewers at Agents4Science. arXiv:2511.15534, 2025.

[6] Anne Chao. Estimating the population size for capture–recapture data with unequal catchability. *Biometrics*, 43(4):783–791, 1987.

[7] Yueh-Han Chen, Jiaxin Wen, and Jan Hendrik Kirchner. Automated researchers can mitigate well-characterized alignment failures. arXiv:2608.28945, 2026.

[8] Tianyu Ding, Aditya Nannapaneni, Bingfan Liu, and Ling Zhang. Autonomous research agents: A survey of AI scientists and the verification gap. arXiv:2608.05179, 2026.

[9] Junyu Guo, Yuchen Fang, Shangding Gu, Costas Spanos, James Demmel, and Javad Lavaei. When context changes: Understanding update failures in LLMs. arXiv:2609.38866, 2026.

[10] Bhaskar Gurram. Auditing automated evaluation, error propagation, and runtime mitigation in tool-using language agents. arXiv:2604.16706, 2026.

[11] Tal Ifargan, Lukas Hafner, Maor Kern, Ori Alcalay, and Roy Kishony. Autonomous LLM-driven research from data to human-verifiable research papers. arXiv:2404.17605, 2024.

[12] Nidhi Jha, Siddharth Chaudhary, and Ajinkya Kulkarni. Paper pilot: A human-in-the-loop expert system for evidence-traceable scientific manuscript generation in applied sciences. arXiv:2608.28596, 2026.

[13] Seungone Kim, Dongkeun Yoon, Kiril Gashteovski, Juyoung Suk, Jinheon Baek, Pranjal Aggarwal, et al. On the limits and opportunities of AI reviewers: Reviewing the reviews of Nature-family papers with 45 expert scientists. arXiv:2605.20668, 2026.

[14] Atsuyuki Miyai, Mashiro Toyooka, Takashi Otonari, Zaiying Zhao, and Kiyoharu Aizawa. Jr. AI scientist and its risk report: Autonomous scientific exploration from a baseline paper. arXiv:2511.04583, 2025.

[15] Atsuyuki Miyai, Mashiro Toyooka, Zaiying Zhao, Kenta Watanabe, Toshihiko Yamasaki, and Kiyoharu Aizawa. Paper reconstruction evaluation: Evaluating presentation and hallucination in AI-written papers. arXiv:2604.01128, 2026.

[16] Yoojin Nam, Jinhoon Jeong, and Namkug Kim. Deterministic integrity gates for LLM-assisted clinical manuscript preparation: An auditable biomedical informatics architecture. arXiv:2606.09500, 2026.

[17] Dang Nguyen, Wanqing Hao, Yanai Elazar, and Chenhao Tan. Benchmarking agentic review systems. arXiv:2606.19749, 2026.

[18] Nhat-Minh Nguyen. Physics is all you need? a case study in physicist-supervised AI development of scientific software. arXiv:2605.30353, 2026.

[19] Roberto I. Ono Filho. Interrupting the loop: Periodic subject changes raise judged surprise and connection in base language models. arXiv:2608.19893; corrected version 2 on Zenodo, doi:10.5281/zenodo.23163693, 2026.

[20] Roberto I. Ono Filho. Moving the mean toward the known good, not beyond it: What inference-time interventions and weight consolidation buy in open-ended generation. arXiv:2608.28886; corrected version 2 on Zenodo, doi:10.5281/zenodo.23163706, 2026.

[21] Roberto I. Ono Filho. Operator packages, proposer strength, and construction-family plateaus in office-scale verified search. arXiv:2609.29636; corrected version 4 on Zenodo, doi:10.5281/zenodo.23163677, 2026.

[22] Roberto I. Ono Filho. What state-space injection buys in open-ended generation, and what it costs. Zenodo, corrected version 4, doi:10.5281/zenodo.23163708, 2026.

[23] Roberto I. Ono Filho. Fine-tuning a language model on its own verified Lean proofs: Transfer, generalisation and saturation across three iterations. Report, Zenodo, doi:10.5281/zenodo.23163729, 2026.

[24] Roberto I. Ono Filho. What a windowed LLM judge cannot see: Provenance, out-of-horizon self-copy, and the whole document. Zenodo, doi:10.5281/zenodo.23163731, 2026.

[25] Tyler S. Orden. How far a wrong claim travels before it is caught. Poster, NeurIPS 2026 Workshop on AI-Native Academia: Authorship, Peer Review, and Conference Governance under AI, 2026. URL https://neurips.cc/virtual/2026/166278. OpenReview Bm1sYpnBgT; accessed 16 October 2026.

[26] Sara Schroter, Nick Black, Stephen Evans, Fiona Godlee, Lyda Osorio, and Richard Smith. What errors do peer reviewers detect, and does training improve their ability to detect them? *Journal of the Royal Society of Medicine*, 101(10):507–514, 2008.

[27] Matthew D. Schwartz. Vibe physics: The AI grad student. Anthropic research blog, 23 March 2026, https://www.anthropic.com/research/vibe-physics; the paper is arXiv:2601.02484, 2026.

[28] Nolan Smyth, Yorguin-Jose Mantilla-Ramos, Pascal Jr Tikeng Notsawo, Saskia Helbling, Alberto Tosato, Mohamed Amine Merzouk, et al. Quantifying overclaiming propensity in frontier LLM agents. arXiv:2609.20812, 2026.

[29] Guijin Son, Jiwoo Hong, Honglu Fan, Heejeong Nam, Hyunwoo Ko, Seungwon Lim, et al. When AI co-scientists fail: SPOT, a benchmark for automated verification of scientific research. arXiv:2505.11855, 2025.

[30] Tae-Eun Song. When does a second model help? cross-model review in LLM verification. arXiv:2610.01471, 2026.

[31] Chupei Wang and Jiaqiu Vince Sun. Unable to forget: Proactive interference reveals working memory limits in LLMs beyond context length. arXiv:2506.08184, 2025.

[32] Sarina Xi, Vishisht Rao, Justin Payan, and Nihar B. Shah. FLAWS: A benchmark for error identification and localization in scientific papers. arXiv:2511.21843, 2025.

[33] Yihan Xia and Taotao Wang. ResearchLoop: An evidence-gated control plane for AI-assisted research. arXiv:2605.28282, 2026.

[34] Chen Zhu, Xiaolu Wang, and Weilong Zhang. (human) attention is (still) all you need: Human oversight makes AI-assisted social science reliable. arXiv:2606.12848, 2026.

A The review of this report

Before deposit, this report was put through rule 3 in rounds (in the archive, the second round is round 1, and so on). The first (fresh reviews by Claude Opus 5.5 and GPT-6.1 Sol) found one wrong count, an overstated causal conclusion and several overbroad phrasings. The second (a referee review with an audit against the records by Claude Fable 5.1, and document reviews by GPT-6.1 Sol and Kimi K3) found, among smaller things, five errors of the kinds this report counts: examples of uncaught errata that had been caught, a check of the matcher described as

done by hand that was the assistant's, a character count given as tokens, drafts called published, and an availability statement that promised raw requests which are not archived; its questions prompted the post hoc analysis of Section 4.3. The third (a new referee review with an audit against the records by a fresh Claude Fable 5.1 agent, and new document reviews by GPT-6.1 Sol and Kimi K3) found that amendments 4–6 had been written after their steps, that the records-level audit lacked some of the records the text said it had, that the notebook itself carried some errata, and smaller things. The fourth (a third fresh referee, Claude Fable 5.1, and new document reviews by GPT-6.1 Sol and Kimi K3) found that the closing sentence of Section 5 contradicted Section 4.3, that a coding we did ourselves stood undisclosed behind a number in the abstract, and that the frozen paper 1 carried rater passages to be redacted before deposit, among smaller things. The fifth (a fourth fresh referee, Claude Fable 5.1, and new document reviews by GPT-6.1 Sol and Kimi K3) found that the capture–recapture occasions departed from the pre-registration's text, that the registered sensitivity analyses with the second coder were missing, and that the headline precision mixed audited errata with new errors that the scored model had helped to confirm, among smaller things. The sixth (a fifth fresh referee and new document reviews by GPT-6.1 Sol and Kimi K3) found that what we had called the notebook's account of why the errors happened was the assistant's own reading, recorded after the pilots, and smaller things, among them a matching amendment the text did not mention. The seventh (a sixth fresh referee and new document reviews by GPT-6.1 Sol and Kimi K3) found a closely related study we had not cited, three omissions counted among the errata nothing catches that no check had been asked about, an availability sentence that did not fit the records-level audit, and smaller things. The eighth (a seventh fresh referee and new document reviews by GPT-6.1 Sol and Kimi K3) found that the matcher had credited document-only reviews with errata through findings that raised a different concern about the same sentence, which led to the strict re-check of Section 3, and that a sentence on the adjudicators' agreement left out their third answer, among smaller things. The ninth (an eighth fresh referee and new document reviews by GPT-6.1 Sol and Kimi K3) found that the strict re-check had not been applied to precision, that the pilots' unfrozen protocol and unvalidated scorer were not stated, that the frozen texts come from a private repository, and a related benchmark we had not cited, among smaller things. The tenth (a ninth fresh referee and new document reviews by GPT-6.1 Sol and Kimi K3) found that recall and precision counted different findings, about half of a review's catches coming through findings it rated low, among smaller things. The eleventh (a tenth fresh referee and new document reviews by GPT-6.1 Sol and Kimi K3) found that the matcher's validation had used easier negatives than the pre-registration's draft described, that one more checker response and the agreement between each pair of raters carried per-rater data to be redacted, and that a sentence on the notebook's size contradicted the size comparison, among smaller things. The twelfth (an eleventh fresh referee and new document reviews by GPT-6.1 Sol and Kimi K3) found that the claim about the human reviewer's five had not been carried through the strict re-check, and smaller things, among them percentages rounded half to even. The thirteenth (a twelfth fresh referee and new document reviews by GPT-6.1 Sol and Kimi K3) found that one of the new errors was an audited erratum the matcher had missed, that the four pilot records were one study, and that one credit surviving the strict re-check rested on a check its checker could not have made, among smaller things. The review stopped there, by the author's decision: every round had found something, the later ones only secondary errors, so this report surely still carries some. A fresh agent then checked only that round's changes, recomputing every new number, and found three small points of wording and consistency, corrected here.

B Registered measures not discussed in the text

The pre-registration also listed recall by document for every check, the overlap between the checks' catches and Chao's estimate by class. Table 4 gives recall by document. The overlaps

(Jaccard, on the primary errata) are 0.66 between the two Opus runs, 0.45 between Opus and GPT-6.1 Sol (either run of each), 0.41 between Opus and the records-level audit, and 0.42 between the twelve reviews and the records-level audit (23 of the 55 errata either caught). Table 5 gives Chao's estimate by class over all occasions, as in Section 4.5; a new error takes the class the Opus adjudicator gave it, and the estimate is unstable for the small classes.

Table 4: Primary errata caught by each check in each document. P1–P4 are papers 1–4, PS the short paper and P6 the draft report; the number of primary errata of each document is under its name.

check	P1	PS	P2	P3	P4	P6
	14	6	16	16	11	17
C1 Claude Opus 5.5, run 1	6	5	4	6	4	3
C1 Claude Opus 5.5, run 2	5	5	6	7	3	4
C1 GPT-6.1 Sol, run 1	7	4	5	4	3	3
C1 GPT-6.1 Sol, run 2	5	3	4	1	2	3
C1 Kimi K3, run 1	3	2	4	4	1	4
C1 Kimi K3, run 2	4	4	4	4	3	3
C1 Mistral Large 3, run 1	1	1	1	1	1	0
C1 Mistral Large 3, run 2	1	0	0	2	0	0
C1 Qwen3-Next-80B, run 1	0	1	0	1	1	0
C1 Qwen3-Next-80B, run 2	1	1	1	1	0	0
C1 DeepSeek V3.2, run 1	0	1	0	0	0	1
C1 DeepSeek V3.2, run 2	0	0	0	1	0	0
C1, all twelve reviews	9	5	11	9	6	8
C2 Opus 5.5 with the records	5	3	6	6	4	6
C3 numbers	1	3	1	1	0	0
C4 citations	0	0	1	0	0	1
all replayed checks	10	6	12	12	7	10

Table 5: Chao's lower-bound estimate by class: errors seen (primary errata and new errors), estimate and the share seen.

class	seen	estimate	seen of estimate
INTERNAL	45	46.3	97%
NUM	29	37.1	78%
SCOPE	39	40.9	95%
META	16	28.2	57%
MIX	9	10.5	86%
TYPO	4	7.0	57%
METHOD	3	4.0	75%
STATUS	2	3.0	67%
PROP	2	3.0	67%
CITE	2	2.0	100%